Batching

pay the fixed cost once for many items

Watch the 4-min video

In one line: batching groups many items into one operation so a fixed overhead (round trip, commit, syscall, GPU launch) is paid once and amortised; it raises throughput but makes items wait, so cap both batch size and wait time; micro-batching processes small batches continuously, and buffered items need a plan for crashes and partial failures.

1 Two thousand round trips a second

Your service writes every click event to Postgres, one `INSERT` at a time. At 2,000 events per second, the database spends most of its time on round trips and commits, not on data. Group those writes, and the same database handles ten times the load.

2 Why it works: amortisation

Every operation has a **fixed overhead**: a network round trip, a transaction commit (an fsync), a system call, a GPU kernel launch. If each item pays that overhead alone, the overhead dominates. Batching groups many items into one operation, so the fixed cost is paid once and spread across the batch: **amortisation**. One commit with 1,000 rows costs about the same fsync as a commit with one row.

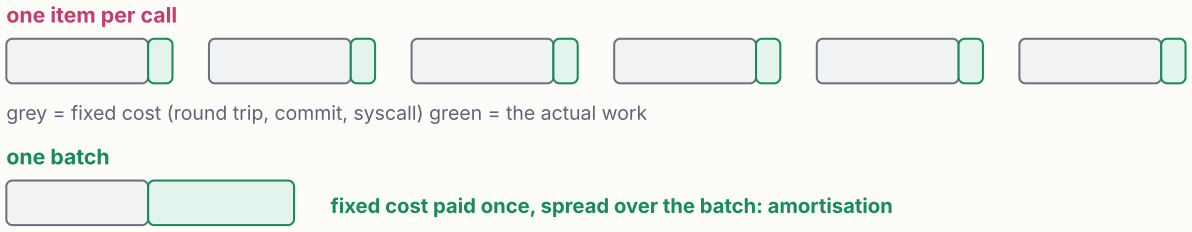


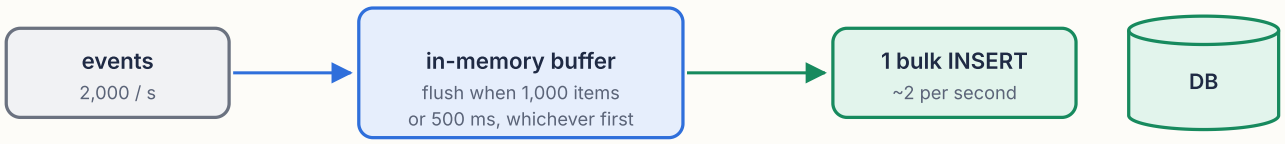
Figure 1. Per-item calls pay the fixed cost every time; a batch pays it once.

3 Where batching shows up

- **Databases**: multi-row `INSERT`, `COPY`, `executemany`, bulk upserts.
- **Redis**: `MGET` / `MSET` and pipelining (many commands, one round trip).
- **Kafka producers**: `linger.ms` and `batch.size` group messages per partition, and compress whole batches.
- **Batch APIs** that accept a list of items.
- **ML inference**: servers batch requests so the GPU processes many inputs in one pass.

4 The cost: waiting

To build a batch, items have to wait. Bigger batches give better throughput, but each item waits longer before it's processed. So set **two limits**: a maximum **batch size** and a maximum **wait time**, and flush when either is reached. Under heavy load batches fill quickly; under light load the timer bounds the delay.



heavy load: batches fill fast (size limit) · light load: the timer bounds the wait (time limit)

Figure 2. A buffer that flushes on size or time, whichever comes first.

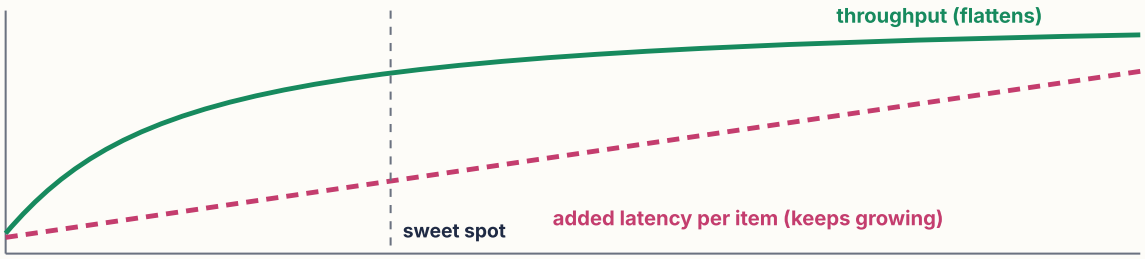


Figure 3. Throughput gains flatten as batches grow, while added latency keeps rising: pick a size inside your latency budget.

5 Micro-batching

Between one item at a time and one huge nightly batch is **micro-batching**: small batches processed continuously, every few hundred milliseconds. Spark Streaming made it famous, and **dynamic batching** in model servers such as Triton and vLLM (continuous batching of tokens) uses the same idea for inference.

WORKED EXAMPLE: CLICK EVENTS

The API appends click events to an in-memory buffer. A background task flushes it every **500 ms**, or when it reaches **1,000 events**, with one bulk insert (`COPY` or a multi-row INSERT). Database round trips drop from 2,000 per second to about **2 per second**. Each event lands at most half a second later, which is fine for analytics.

6 Risks

- **Crash loss**: items in an in-memory buffer are lost if the process dies. Put durable data on a queue first, or accept the loss for analytics.
- **Partial failure**: one bad item can fail the whole batch. Validate first, or retry the rest and send the bad item to a dead-letter queue.
- **Big batches** cause memory spikes, long locks and replication lag. Keep them moderate (hundreds to a few thousand rows).
- **Measure**: tune size and wait time against your latency budget.

COMMON MISTAKES

- A size limit with no time limit: at night, items wait forever.
- Batching user-facing writes the user must see immediately.
- Not flushing the buffer on graceful shutdown.

7 Key terms

Batching: many items in one operation

Pipelining: send many commands without waiting for each reply

Micro-batching: small batches, processed continuously

Amortisation: spreading a fixed cost over many items

linger.ms: Kafka producer wait time to fill a batch

Dynamic batching: an inference server grouping requests on the fly

8 Interview questions

Q1. What are the latency trade-offs of batching? BASIC

Batching raises throughput by amortising fixed overhead, but each item waits for the batch to fill, so you cap both the batch size and the wait time to bound the added latency.

Q2. What is micro-batching? BASIC

Processing small batches continuously, every few hundred milliseconds, to balance throughput and freshness.

Q3. Your service makes 50 Redis GETs per request. How do you speed it up? SCENARIO

Batch them with MGET or a pipeline so they share one round trip; at 0.5 ms per round trip that saves about 25 ms per request.

Q4. Design ingestion of 50,000 events per second into a database. SCENARIO

Put events on a durable queue (Kafka), have consumers read in batches, and write with bulk inserts or COPY every few hundred milliseconds or few thousand rows, with idempotent upserts for retries.

Q5. Why does batching help GPU inference so much? TRICKY

A GPU pass has large fixed costs (kernel launches, moving weights through memory); processing many inputs at once uses its parallel cores fully, so throughput rises many times while per-request latency rises a little.

Q6. One row in a 1,000-row batch violates a constraint. What happens and what do you do? TRICKY

The whole transaction fails. Validate beforehand, use ON CONFLICT handling, or split and retry (binary split) to isolate the bad row and send it to a dead-letter queue.

Next: SD-J-007 · Compression backendroadmap.com